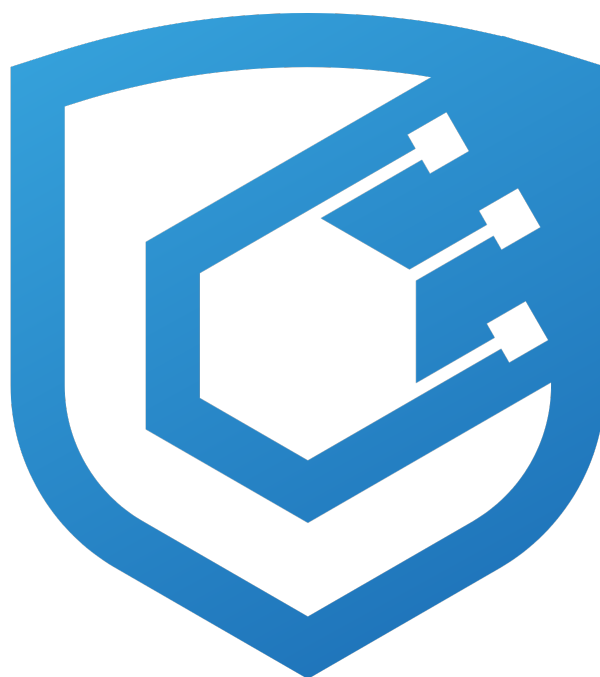




Avant RequestManagerV2 Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[Dacian](#)

[JuliusRaynaldi](#) (Formal Verification)

June 24, 2026

Contents

1	About Cyfrin	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	2
6	Executive Summary	3
6.1	Centralization Risks	3
6.1.1	Idempotency Keys	3
6.1.2	Multiple Tokens, Single Pricing Function	3
6.1.3	Inherent Admin Powers	3
6.1.4	Privileged Attack Paths	3
6.2	Defensive Improvements	4
7	Findings	6
7.1	Low Risk	6
7.1.1	Missing input validation in admin setters across PriceStorage, RequestsManagerV2	6
7.2	Informational	7
7.2.1	RequestsManagerV2::cancelBurn reverts during a pause longer than burnCancelWindow, breaking the documented pause-cancellation invariant	7
7.2.2	SimpleToken::idempotentMint, idempotentBurn write the idempotency flag after the body executes instead of before	7
7.2.3	README.md documentation mismatches: stale and inaccurate references disagree with the code and docs	8
7.3	Gas Optimization	10
7.3.1	RequestsManagerV2 cache storage slots to prevent identical storage reads	10

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Avant's `RequestsManagerV2` is a request-based mint and burn manager for a stablecoin-style issue token. Users request a:

- mint by depositing an allowlisted ERC20
- burn by returning previously minted issue tokens

A trusted backend completes each request on-chain, settling at a price read from a separate `PriceStorage` oracle contract and applying configurable mint and burn fees. Deposited collateral is custodied in the manager and routed to a treasury address on mint completion.

The issue token is the upgradeable `SimpleToken` which together with `PriceStorage` are existing contracts already deployed as proxies that are reused unchanged at the cutover from the previous `RequestsManager` to `RequestsManagerV2`.

Privileged operations are split across three roles: * `DEFAULT_ADMIN_ROLE` - custodied by a multi-signature wallet, owns the parameter setters, treasury, emergency withdrawal, admin cancels and can unpause * `SERVICE_ROLE` - off-chain backend service, publishes the settlement price and completes every mint and burn * `PAUSER_ROLE` can pause the contract and may be co-located with the backend

5 Audit Scope

The audit scope was limited to:

```
src/RequestsManagerV2.sol
src/interfaces/IRequestsManagerV2.sol
src/PriceStorage.sol
src/SimpleToken.sol
src/interfaces/ISimpleToken.sol
src/interfaces/IPriceStorage.sol
```

6 Executive Summary

Over the course of 5 days, the Cyfrin team conducted an audit on the [RequestManagerV2](#) code provided by [Avant](#). The findings consist of 1 Low severity issue with the remainder being informational and gas optimizations.

6.1 Centralization Risks

6.1.1 Idempotency Keys

The V1 `RequestsManager` takes an arbitrary `_idempotencyKey` as input while the V2 hardcodes a string literal plus the request `_id` for the `_idempotencyKey`.

There is nothing on-chain to prevent `SERVICE_ROLE` from calling the V1 `RequestsManager` with a V2-style `_idempotencyKey`, which would brick subsequent operations once V2 is live. This needs to be correctly managed off-chain such that `SERVICE_ROLE` does not use V2-style `_idempotencyKey` with V1 `RequestsManager`.

6.1.2 Multiple Tokens, Single Pricing Function

`RequestsManagerV2` supports multiple allow-listed deposit/withdraw tokens which all use the same price from `PriceStorage`. Hence the admin must be careful to validate that the allowed tokens can all be priced the same against the issue token. There is also the risk that if one of the configured tokens depegs, this could be abused during the depeg window. For example if two 18-decimal dollar-pegged tokens A and B are both allowlisted (the allowlist enforces `decimals() == 18`, so 6-decimal stablecoins such as USDC/USDT cannot be used directly) and A depegs to \$0.50 while B holds at \$1.00, a user could deposit 1000 A (real value \$500) to mint issue tokens credited at the single global \$1.00 price, then redeem 1000 B (real value \$1000). This can be mitigated off-chain by having the off-chain services monitor for depegs and pause `RequestsManagerV2` if a depeg is detected, which prevents any deposits using depegged tokens.

6.1.3 Inherent Admin Powers

- `DEFAULT_ADMIN_ROLE` controls every economic and temporal parameter, including the mint and burn fees, the mint and burn request TTLs, the burn cancel window, and the `PriceStorage` bound percentages; it can also repoint the treasury, force-cancel any pending mint or burn, add and remove allowed deposit tokens, and sweep all contract funds through `emergencyWithdraw`
- `SERVICE_ROLE` publishes the settlement price and completes every mint and burn; via `RequestsManagerV2`'s completion path it triggers settlement only and cannot choose the minted or burned amounts, which are derived on-chain. This guarantee does **not** extend to the issue token itself - see the keyless-overload path under Privileged Attack Paths. It can also directly mint/burn tokens via `SimpleToken::mint,burn`
- `PAUSER_ROLE` can pause the contract, but only `DEFAULT_ADMIN_ROLE` can unpause
- The admin role is custodied by a multi-signature wallet, while `SERVICE_ROLE` is the off-chain backend service and `PAUSER_ROLE` may be co-located with it, so role separation is operational rather than enforced by distinct cryptographic controls

6.1.4 Privileged Attack Paths

- The issue token `SimpleToken` exposes keyless `mint(address,uint256) / burn(address,uint256)` overloads that any holder of `SERVICE_ROLE` on the issue token can call to mint or burn arbitrary amounts, with no idempotency key and no on-chain amount derivation. `RequestsManagerV2` itself only ever calls the keyed overloads, but during the V1→V2 migration both managers hold `SERVICE_ROLE` on the shared issue token, so a compromise of any issue-token `SERVICE_ROLE` holder can mint unlimited issue tokens regardless of escrowed collateral. `SERVICE_ROLE` grants on the issue token should be minimised and time-boxed (`AUDIT_NOTES.md §4.9`)

- `SERVICE_ROLE` can move the global settlement price arbitrarily far within a single block by chaining many individually-bounded `setPrice` steps, because the per-update bound is enforced per call rather than per unit of time
- Oracle freshness is not enforced on-chain: `lastPrice()`'s timestamp is ignored, so settlement can occur against an arbitrarily stale price. The per-update bounds cap how far each *published* update may move, but they do **not** cap staleness loss: if the oracle stops updating while the real asset price diverges, mints and burns keep settling at the frozen price until the backend pauses, and on recovery the `lowerBoundPercentage` can even slow the on-chain price's downward correction, prolonging the mispricing. Worst-case economic loss is therefore a function of real-world volatility and the backend's pause latency rather than the configured bounds alone; quantifying it requires the production bound values (TBD at the time of review) together with the operational pause cadence. Circuit-breaking a broken or stale oracle is an off-chain (pause) responsibility, not an on-chain guarantee
- `PriceStorage::setLowerBoundPercentage`, `setUpperBoundPercentage` accept any value in $(0, 1e18]$, with neither a sane floor nor a sane cap which can be abused by `DEFAULT_ADMIN_ROLE` in a number of ways (see finding L-1)
- An admin can change a request's economics after the user's funds are already escrowed: raising the live-read mint fee reduces a pending mint's output, and changing the request TTLs can instantly expire pending requests or widen the no-escape exposure window of in-flight burns
- Reducing the burn cancel window retroactively closes the self-cancel escape on already-open burns, stranding redeemers who can no longer abort an unfavorable settlement
- Removing an allowed token does not stop in-flight requests denominated in that token from settling, because completion never re-checks the allowlist (see finding L-2)
- Mint settlement for a deposit token can be denied if that token's issuer blocklists the treasury address, until the admin repoints the treasury
- The 1-day `AccessControlDefaultAdminRules` delay gates only the transfer of `DEFAULT_ADMIN_ROLE` to a new address; it doesn't gate direct invocation of admin-only functions. A compromised admin key already holds the role and can call `emergencyWithdraw` immediately, draining escrowed mint deposits so that subsequent `cancelMint` calls revert on transfer. This is mitigated off-chain using a multi-signature wallet with transaction authorization policies

6.2 Defensive Improvements

- Deployer private key is loaded from a plaintext env var - (`script/DeployV2.s.sol`, `script/Deployment.s.sol`). The scripts read `PRIVATE_KEY` directly from the environment. Use Foundry's private key store for improved [private key safety](#)
- Hardcoded per-network addresses with no chain-id assertion - (`script/DeployV2.s.sol`). `DeployV2` binds to fixed per-product, per-network addresses without asserting the running chain id, risking a wrong-network deployment. Assert `block.chainid` against the intended network before deploying
- `PriceStorage::setPrice` should prevent itself from being called multiple times in the same block, to prevent arbitrary price movements. Ideally there would be a fixed time of say 30-60 seconds enforced between each call
- After `RequestManagerV2` is deployed and migration is completed, ideally `SimpleToken` would be upgraded to remove the `mint`, `burn` functions without `_idempotencyKey` and the remaining `mint`, `burn` functions would be gated to only be callable by the deployed `RequestManagerV2`. This way even if `SERVICE_ROLE` would be compromised it couldn't be used for infinite mints/burns

Summary

Project Name	RequestManagerV2
Repository	Avant-Contracts-Max
Commit	0738f33ebc96...
Fix Commit	ec30a6ccba0d...
Audit Timeline	June 15th - June 19th, 2026
Methods	Manual Review, Formal Verification

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	1
Informational	3
Gas Optimizations	1
Total Issues	5

Summary of Findings

[L-1] Missing input validation in admin setters across PriceStorage, RequestsManagerV2	Acknowledged
[I-1] RequestsManagerV2::cancelBurn reverts during a pause longer than burnCancelWindow, breaking the documented pause-cancellation invariant	Resolved
[I-2] SimpleToken::idempotentMint, idempotentBurn write the idempotency flag after the body executes instead of before	Resolved
[I-3] README.md documentation mismatches: stale and inaccurate references disagree with the code and docs	Resolved
[G-1] RequestsManagerV2 cache storage slots to prevent identical storage reads	Resolved

7 Findings

7.1 Low Risk

7.1.1 Missing input validation in admin setters across PriceStorage, RequestsManagerV2

Description: Several admin-only setters validate only a trivial degenerate input and accept other values that are syntactically valid but break pricing or settlement. Each is independently low-severity (admin recovers by re-calling the setter), but they share one root cause: the accepted input domain is wider than the safe domain.

1. `PriceStorage::setLowerBoundPercentage, setUpperBoundPercentage` accept any value in $(0, 1e18]$, with neither a sane floor nor a sane cap.

Both setters reject only 0 and values strictly greater than `BOUND_PERCENTAGE_DENOMINATOR` ($1e18$) (`src/PriceStorage.sol:56-72`). The per-update bounds in `setPrice` are computed as $\text{lastPriceValue} \pm \text{lastPriceValue} * \text{percentage} / 1e18$ (`src/PriceStorage.sol:38-44`). At the high end, a bound at or near $1e18$ drives the lower bound to 0, so a single `setPrice` can crash the price to 1 wei and the next `completeMint` mints a vastly inflated amount.

At the low end, a percentage small enough that the band $\text{lastPriceValue} * \text{percentage} / 1e18$ rounds down to zero (or, at the canonical $1e18$ price scale, to a ~ 1 -wei width that rejects every realistic update) collapses the band to `upperBound == lowerBound == lastPriceValue`; every genuine price change then reverts `InvalidPriceRange`.

Because the per-update band check is computed from `lastPrice` and applies to every key, even a fresh key cannot publish a value outside the band (write-once-per-key at `src/PriceStorage.sol:34` only blocks overwriting an existing key), so the oracle is frozen until the admin widens the bound.

2. `RequestsManagerV2::setTreasury` and the constructor accept the self-address `address(this)`.

Both validate only that the treasury is non-zero (`src/RequestsManagerV2.sol:139-143`, constructor at `src/RequestsManagerV2.sol:113`); neither rejects `address(this)`.

With `treasuryAddress == address(this)`, the `completeMint` deposit forward becomes a self-transfer no-op that commingles deposits with escrow (`src/RequestsManagerV2.sol:298`), and `completeBurn` requires a self-allowance that is normally absent, so every burn settlement reverts on `safeTransferFrom` (`src/RequestsManagerV2.sol:425`) until the treasury is re-pointed.

Recommended Mitigation:

- `PriceStorage::setLowerBoundPercentage, setUpperBoundPercentage`: bound each percentage within a sane closed band rather than $(0, 1e18]$ - enforce both a floor large enough that the per-update band cannot round down to a sub-update width at the protocol's price scale, and a cap materially below $1e18$ (e.g. $0.5e18$) consistent with the intended 5% / 33% regime. `initialize` routes through these setters, so the fix covers initialization.
- `RequestsManagerV2::setTreasury` and constructor: reject `address(this)` in addition to the existing non-zero check (e.g. `revert when _treasuryAddress == address(this)`).

Avant: Acknowledged; `PriceStorage` is pre-existing and deployed immutably but we will note this for future deployments. `RequestsManagerV2::setTreasury` we accept as a deployment-time admin responsibility.

7.2 Informational

7.2.1 `RequestsManagerV2::cancelBurn` reverts during a pause longer than `burnCancelWindow`, breaking the documented pause-cancellation invariant

Description: The pause NatSpec states that *"cancellation paths stay enabled so pending requests can always be unwound while paused"* (`src/RequestsManagerV2.sol:155-156`). This holds for `cancelMint`, which has no window or TTL gate and is always callable while `CREATED`. It does not hold for `cancelBurn`: that function reverts `BurnCancelWindowClosed` once `block.timestamp > createdAt + burnCancelWindow` (`src/RequestsManagerV2.sol:362-369`). The cancel window is measured against wall-clock `block.timestamp` and keeps elapsing during a pause - the pause does not freeze time. Because `completeBurn` carries `whenNotPaused`, it is unavailable during the pause as well.

If the contract is paused continuously for longer than a pending burn's remaining cancel window (deployed `burnCancelWindow` is 2 days), the provider can neither complete (the contract is paused) nor cancel (the window has closed). The documented *"always be unwound while paused"* guarantee is false for the burn path: the provider's only self-service exit is gone precisely while the protocol is paused, which is when a user most wants out. The escrowed issue tokens are recoverable only through the admin-gated `adminCancelBurn`, which has no window or pause gate.

Recommended Mitigation: Update the NatSpec above `RequestsManagerV2::pause` to correct the false claim that "pending requests can always be unwound while paused" - this is only true for mints but not burns.

Avant: Fixed in commit [f7d9275](#).

Cyfrin: Verified.

7.2.2 `SimpleToken::idempotentMint`, `idempotentBurn` write the idempotency flag after the body executes instead of before

Description: The keyed `SimpleToken::mint` and `SimpleToken::burn` overloads guard against idempotency-key reuse with the `idempotentMint` and `idempotentBurn` modifiers (`src/SimpleToken.sol:20-34`). Both modifiers run the function body at `_`; and only afterwards record the key as used:

```
modifier idempotentMint(bytes32 _idempotencyKey) {
    if (mintIds[_idempotencyKey]) {
        revert IdempotencyKeyAlreadyExist(_idempotencyKey);
    }
    _;
    mintIds[_idempotencyKey] = true; // body runs first
}                                     // effect applied afterwards
```

This inverts the checks-effects-interactions ordering: the effect (recording the key in `mintIds` / `burnIds`) is applied after the interaction in the body (the `_mint` / `_burn` call) rather than before it. The canonical idempotency guard records the key as consumed before executing the body, so that any re-entrant call observes the key as already used and reverts.

With the current code the ordering is harmless. Each keyed overload's body is a single `_mint` / `_burn` call, and in OpenZeppelin Contracts Upgradeable v5.3.0 those route through `ERC20Upgradeable._update` (out of scope, traced for context), which only mutates balances and emits `Transfer`. It makes no external call and invokes no token-transfer or receiver hook - the `_beforeTokenTransfer` / `_afterTokenTransfer` hooks present in OZ v4 were removed in v5. `SimpleToken` does not override `_update`. Control therefore never leaves the contract between the key check and the key write, so a single key cannot be consumed twice within one transaction and no double-mint or double-burn is reachable today.

Here there is no current impact however this is also not good practice; writing the effect before the body removes the hazard unconditionally and costs nothing.

Recommended Mitigation: Apply the effect before running the body so the guard holds regardless of what the body later does:

```
modifier idempotentMint(bytes32 _idempotencyKey) {
```



```

    if (mintIds[_idempotencyKey]) {
        revert IdempotencyKeyAlreadyExist(_idempotencyKey);
    }
    mintIds[_idempotencyKey] = true;
    -;
}

```

Apply the same reordering to `idempotentBurn`.

Avant: Fixed in commit [ff7803c](#).

Cyfrin: Verified.

7.2.3 README.md documentation mismatches: stale and inaccurate references disagree with the code and docs

Description: README.md contains several statements that disagree with the in-scope contract source and the supplied project docs (AUDIT_NOTES.md, RFP-RequestsManagerV2-external-audit.md). The inaccuracies mis-state burn pricing, fees, the request-counter offset, the price bounds, and admin powers in ways a reader or integrator would act on.

1. README.md:82, 88 - burn settlement is described as using only the stored request-time price.

The README states `completeBurn` "uses this stored price, making the withdrawal amount immune to post-request price changes" (README.md:82) and that burn prices "are locked at request time and cannot change" (README.md:88). The code settles at `min(lockedPrice, currentPrice)` (src/RequestsManagerV2.sol:406-409): the stored price is a ceiling only, so a price fall after the request lowers the payout. AUDIT_NOTES.md:71 and RFP-RequestsManagerV2-external-audit.md:83 both state the redeemer bears a NAV fall.

Recommended: state that burns settle at `min(lockedPrice, currentPrice)` - the request-time price is a ceiling, not a fixed settlement price, and a price decrease is borne by the redeemer.

2. README.md:145 - the burn formula omits the `min(lockedPrice, currentPrice)`.

The formula table gives `withdrawalAmount = (burnAmount * price / PRECISION) * (PRECISION - burnFee) / PRECISION` with a generic price. The code uses `price = min(lockedPrice, currentPrice)` and the locked `request.fee`, not the live `burnFee` (src/RequestsManagerV2.sol:409, 412-413).

Recommended: replace `price` with `min(lockedPrice, currentPrice)` and note the fee is the value locked at request time.

3. README.md:125 - `setBurnFee` is described as retroactive on pending burns.

The trust model lists "Change fees retroactively on pending burns (`setBurnFee`)". The code locks the burn fee into the `BurnRequest` at request time (src/RequestsManagerV2.sol:322, 331) and `completeBurn` consumes `request.fee` (src/RequestsManagerV2.sol:403); the `setBurnFee` NatSpec states it applies "for new requests; pending burns keep the fee locked at request time" (src/RequestsManagerV2.sol:196-197). The RFP records the retroactive-burn-fee issue as fixed (RFP-RequestsManagerV2-external-audit.md:273). The live fee is the mint fee, not the burn fee.

Recommended: remove the retroactive-burn-fee claim; the only live fee is `setMintFee`, applied at completion.

4. README.md:118 - the V2 request-counter start value is wrong.

The README says counters start at 10,000 and that V1 has "fewer than 10,000 existing orders". The code initializes both counters to `INITIAL_COUNTER = 100_000` (src/RequestsManagerV2.sol:47, 127-128); RFP-RequestsManagerV2-external-audit.md:121 and AUDIT_NOTES.md:83 both state 100,000.

Recommended: correct the value to 100,000 and update the dependent order-count statement.

5. README.md:133, 180 - the price bounds are presented as fixed guarantees.

The README implies the bounds are enforced at the +5% upper / -33% lower figures (*"each update is constrained to"* those values). They are settable parameters: `PriceStorage::setUpperBoundPercentage`, `setLowerBoundPercentage` accept any value in `(0, 1e18]` (`src/PriceStorage.sol:56-72`), and the RFP notes the figures derive from test configuration with production values TBD (`RFP-RequestsManagerV2-external-audit.md:145`).

Recommended: describe the bounds as configurable parameters rather than presenting the test-config values as enforced guarantees.

6. `README.md:170` - the role-permissions table omits two admin setters.

The table lists `setBurnRequestTTL` but not `setMintRequestTTL` (`src/RequestsManagerV2.sol:174`) or `setBurnCancelWindow` (`src/RequestsManagerV2.sol:183`), both `DEFAULT_ADMIN_ROLE` setters present in the code.

Recommended: add `setMintRequestTTL` and `setBurnCancelWindow` to the `DEFAULT_ADMIN_ROLE` row.

7. `README.md:92-104` - the "New features" section omits the mint TTL and the burn cancel window.

The section documents the burn TTL but not `mintRequestTTL` (enforced in `completeMint`, `src/RequestsManagerV2.sol:278`) or `burnCancelWindow` (enforced in `cancelBurn`, `src/RequestsManagerV2.sol:367`), both new V2 mechanisms.

Recommended: document `mintRequestTTL` and `burnCancelWindow` alongside the existing burn TTL entry.

Avant: Fixed in commit [a97fcf](#).

Cyfrin: Verified.

7.3 Gas Optimization

7.3.1 RequestsManagerV2 cache storage slots to prevent identical storage reads

Description: Several functions in RequestsManagerV2 read the same storage slot more than once within a single call even though the value does not change between reads. With the optimizer enabled but `via_ir` off, it does not dedupe same-slot reads across statement boundaries or around an intervening external call (a `safeTransfer`, `mint`, or `burn` forces a re-read because storage could have changed during the call), so each becomes a separate warm SLOAD (~100 gas). Caching each value into a local once and reusing it removes the redundant read:

1. `RequestsManagerV2::completeMint` - reads `mintFee` twice, in the fee calculation at `src/RequestsManagerV2.sol:291` and the `MintRequestCompleted` emit at `src/RequestsManagerV2.sol:304`, with two external calls in between. Cache `uint64 fee = mintFee;`, mirroring `completeBurn` which already caches its per-request fee.
2. `RequestsManagerV2::adminCancelMint` - reads `request.provider` twice, at the `safeTransfer` (`src/RequestsManagerV2.sol:267`) and the emit (`src/RequestsManagerV2.sol:269`); the external transfer between them forces the second SLOAD. Cache `address provider = request.provider;` before the transfer.
3. `RequestsManagerV2::adminCancelBurn` - reads `request.provider` twice, at the `safeTransfer` (`src/RequestsManagerV2.sol:384`) and the emit (`src/RequestsManagerV2.sol:386`). Cache before the transfer.
4. `RequestsManagerV2::requestMint` - reads `mintRequestsCounter` at `src/RequestsManagerV2.sol:216` then re-reads it via `mintRequestsCounter++` at `src/RequestsManagerV2.sol:226`. The pre-increment value is already held in the local id.
5. `RequestsManagerV2::requestBurn` - reads `burnRequestsCounter` at `src/RequestsManagerV2.sol:325` then re-reads it via `burnRequestsCounter++` at `src/RequestsManagerV2.sol:337`. Same as above.

Recommended Mitigation: Cache each repeated read into a local before its first use and reuse the local. For the request counters, assign the incremented value directly instead of re-reading the slot:

```
unchecked {  
    mintRequestsCounter = id + 1;  
}
```

For the provider re-reads, load the value before the external transfer so the post-call SLOAD is avoided:

```
address provider = request.provider;  
request.state = State.CANCELLED;  
IERC20(request.token).safeTransfer(provider, request.amount);  
emit MintRequestAdminCancelled(_id, provider, msg.sender);
```

Avant: Fixed in commit [96e36b5](#).

Cyfrin: Verified.